



Escuela de Administración de Tecnología de Información

TI2201- Programación Orientada a Objetos

Docente: Luis Javier Chavarría Sanchez

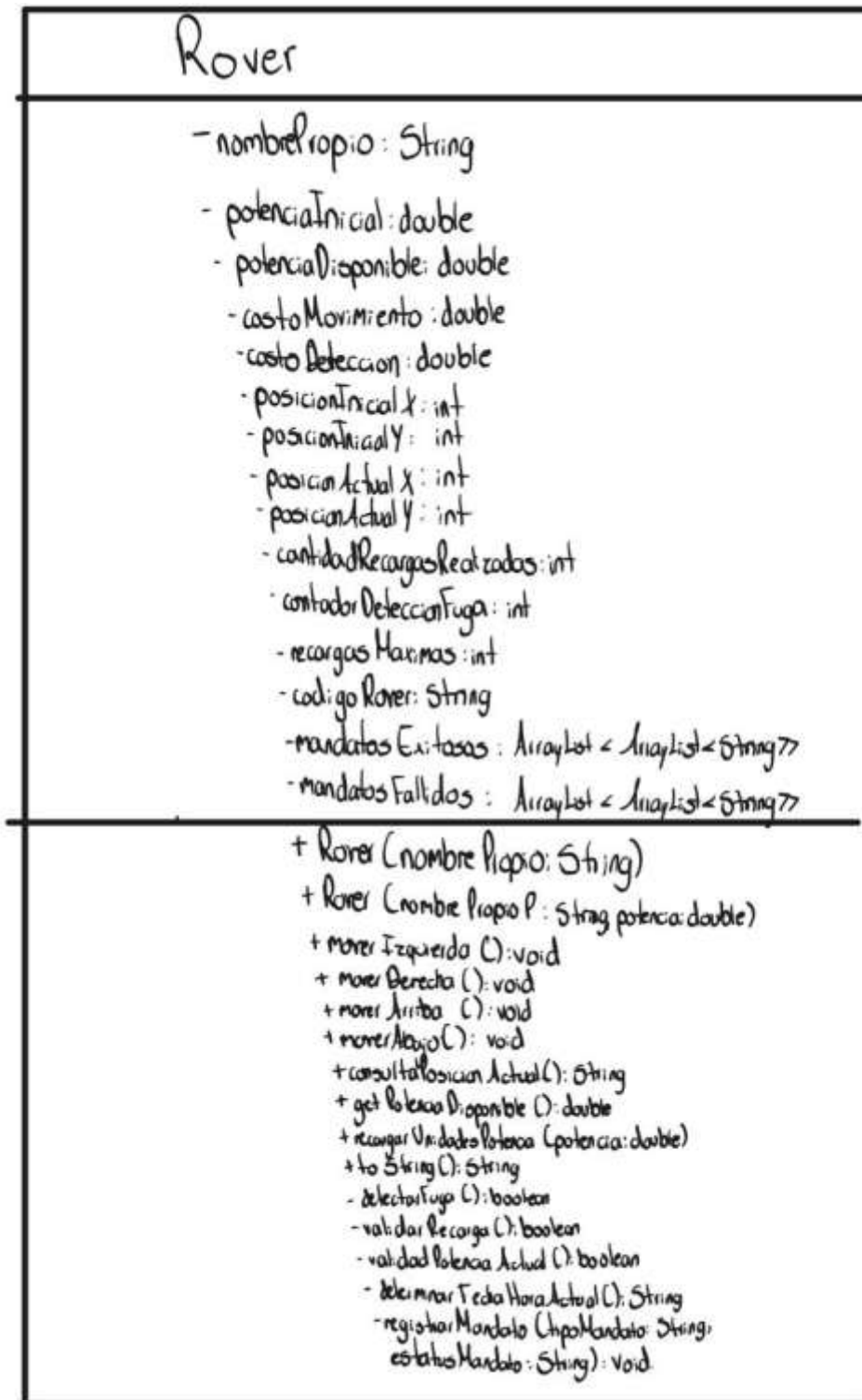
AA3

Elaborado por:
Dennise Arias, 2022040517

Fecha
26 de Agosto del 2026

Cartago

A. Diagrama de clases con detalles clase Rover:



B. Programación de la clase Rover:

```
import java.text.DateFormat;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Random;

public class Rover {
    private String nombrePropio;
    private double potencialInicial;
    private double potenciaDisponible;
    private int posicionInicialX;
    private int posicionInicialY;
    private int posicionActualX;
    private int posicionActualY;
    private int cantidadRecargasRealizadas;
    private int contadorDeteccionesFuga;
    private List<List<String>> mandatosExitosos;
    private List<List<String>> mandatosFallidos;
    private double costoMovimiento;
    private double costoDeteccion;
    private int recargasMaximas;
    private String codigoRover;

    public Rover(String nombrePropio) {
        this(nombrePropio, 100.0);
    }

    public Rover(String nombrePropioP, double potencia) {
        this.nombrePropio = nombrePropioP;
        potencialInicial = potencia;
        potenciaDisponible = potencia;
        posicionInicialX = 0;
        posicionInicialY = 0;
        posicionActualX = posicionInicialX;
        posicionActualY = posicionInicialY;
        cantidadRecargasRealizadas = 0;
        contadorDeteccionesFuga = 0;
        mandatosExitosos = new ArrayList<>();
        mandatosFallidos = new ArrayList<>();
        costoMovimiento = 0.50;
        costoDeteccion = 0.25;
        recargasMaximas = 5;
        codigoRover = "RVR-" + System.currentTimeMillis() % 100000;
```

```
}

public void moverIzquierda() {
    if (validarPotenciaActual()) {
        if (!detectarFuga()) {
            posicionActualX -= 1;
            potenciaDisponibile -= costoMovimiento;
            registrarMandato("Desplazamiento Izquierda", "Posible");
        } else {
            registrarMandato("Desplazamiento Izquierda", "No posible: fuga detectada");
        }
    } else {
        registrarMandato("Desplazamiento Izquierda", "No posible: potencia insuficiente");
    }
}

public void moverDerecha() {
    if (validarPotenciaActual()) {
        if (!detectarFuga()) {
            posicionActualX += 1;
            potenciaDisponibile -= costoMovimiento;
            registrarMandato("Desplazamiento Derecha", "Posible");
        } else {
            registrarMandato("Desplazamiento Derecha", "No posible: fuga detectada");
        }
    } else {
        registrarMandato("Desplazamiento Derecha", "No posible: potencia insuficiente");
    }
}

public void moverArriba() {
    if (validarPotenciaActual()) {
        if (!detectarFuga()) {
            posicionActualY += 1;
            potenciaDisponibile -= costoMovimiento;
            registrarMandato("Desplazamiento Arriba", "Posible");
        } else {
            registrarMandato("Desplazamiento Arriba", "No posible: fuga detectada");
        }
    } else {
        registrarMandato("Desplazamiento Arriba", "No posible: potencia insuficiente");
    }
}

public void moverAbajo() {
    if (validarPotenciaActual()) {
        if (!detectarFuga()) {
```

```
        posicionActualY -= 1;
        potenciaDisponible -= costoMovimiento;
        registrarMandato("Desplazamiento Abajo", "Posible");
    } else {
        registrarMandato("Desplazamiento Abajo", "No posible: fuga detectada");
    }
} else {
    registrarMandato("Desplazamiento Abajo", "No posible: potencia insuficiente");
}
}

private boolean detectarFuga() {
    contadorDeteccionesFuga++;
    potenciaDisponible -= costoDeteccion;
    Random random = new Random();
    return random.nextDouble() >= 0.5;
}

public String consultarPosicionActual() {
    return "Posición actual (x,y): " + posicionActualX + ", " + posicionActualY;
}

public double getPotenciaDisponible() {
    return potenciaDisponible;
}

public void recargarUnidadesPotencia(double potencia) {
    if (validarRecarga()) {
        potenciaDisponible += potencia;
        cantidadRecargasRealizadas++;
        registrarMandato("Recarga (" + potencia + ")", "Posible");
    } else {
        registrarMandato("Recarga (" + potencia + ")", "No posible: recargas agotadas");
    }
}

private boolean validarRecarga() {
    return (cantidadRecargasRealizadas < recargasMaximas) ? true : false;
}

private boolean validarPotenciaActual() {
    double costoMinimo = costoMovimiento + costoDeteccion;
    return potenciaDisponible >= costoMinimo;
}

private String determinarFechaHoraActual() {
    Date fecha = new Date(System.currentTimeMillis());
}
```

```

        DateFormat formatoFecha = new SimpleDateFormat("dd/MM/yy HH:mm:ss");
        return formatoFecha.format(fecha);
    }

    private void registrarMandato(String tipoMandato, String estatusMandato) {
        ArrayList<String> mandato = new ArrayList<>();
        mandato.add(tipoMandato);
        mandato.add(estatusMandato);
        mandato.add(determinarFechaHoraActual());
        if ("Posible".compareTo(estatusMandato) == 0) {
            mandatosExitosos.add(mandato);
        } else {
            mandatosFallidos.add(mandato);
        }
    }

    @Override
    public String toString() {
        String msg = "";
        msg += "===== Ficha del Rover =====\n";
        msg += "Código: " + codigoRover + "\n";
        msg += "Nombre: " + nombrePropio + "\n";
        msg += "Potencia (inicial/disponible): " + String.format("%.2f / %.2f", potencialInicial,
potenciaDisponible) + "\n";
        msg += "Posición (inicial → actual): (" + posicionInicialX + "," + posicionInicialY + ") → (" +
posicionActualX + "," + posicionActualY + ")\n";
        msg += "Costos (mov/detección): " + String.format("%.2f / %.2f", costoMovimiento,
costoDeteccion) + "\n";
        msg += "Recargas (realizadas/máximas): " + cantidadRecargasRealizadas + "/" +
recargasMaximas + "\n";
        msg += "Detecciones de fuga realizadas: " + contadorDeteccionesFuga + "\n";
        msg += "=====\n\n";

        msg += "---- Registro de Mandatos EXITOSOS ----\n";
        msg += String.format(" %-4s %-17s %-30s %-20s\n", "Nº", "Fecha", "Mandato", "Estado");
        for (int i = 0; i < mandatosExitosos.size(); i++) {
            List<String> m = mandatosExitosos.get(i);
            String tipo = (m.size() > 0) ? m.get(0) : "";
            String estado = (m.size() > 1) ? m.get(1) : "";
            String fecha = (m.size() > 2) ? m.get(2) : "";
            msg += String.format(" %-4d %-17s %-30s %-20s\n", (i + 1), fecha, tipo, estado);
        }
        if (mandatosExitosos.isEmpty()) {
            msg += " (sin registros)\n";
        }

        msg += "\n";
    }

```

```
msg += "---- Registro de Mandatos FALLIDOS ----\n";
msg += String.format(" %-4s %-17s %-30s %-20s%n", "N°", "Fecha", "Mandato", "Estado");
for (int i = 0; i < mandatosFallidos.size(); i++) {
    List<String> m = mandatosFallidos.get(i);
    String tipo = (m.size() > 0) ? m.get(0) : "";
    String estado = (m.size() > 1) ? m.get(1) : "";
    String fecha = (m.size() > 2) ? m.get(2) : "";
    msg += String.format(" %-4d %-17s %-30s %-20s%n", (i + 1), fecha, tipo, estado);
}
if (mandatosFallidos.isEmpty()) {
    msg += " (sin registros)\n";
}
return msg;
}
}
```